Learning Association-Line Transformations over Autosegmental Graphs

1. Introduction

This paper presents a method for learning transformations over multi-linear structures, with a particular focus on those structures are synchronized in time. Multi-linear structures are widely used in linguistics in phonological theory in particular, and are known as *Autosegmental Representations* (ARs Clements and Goldsmith, 1984a; Goldsmith, 1976; Hayes, 1980; Sagey, 1986; Jardine, 2017). ARs are graph structures which consist of one or more sequences called 'tiers' and cross-tier association relations. This work focuses on a specific but important type of transformation where the association lines are modified. We formalize these transformations as graph transductions (Courcelle, 1994; Engelfriet and Hoogeboom, 2001; Courcelle and Engelfriet, 2012) over ARs (Jardine and Heinz, 2015; Jardine, 2017; Strother-Garcia, 2019) and use a bottom-up factor inference method (Chandlee et al., 2019; Rawski, 2021; Yolyan, 2025a) to infer which association lines are preserved, deleted, or newly introduced from observed input-output pairs.

Multi-linear structures have been widely used in linguistic theory, most notably in autosegmental phonology, to represent different levels of representations that are coordinated in time, including tone,[1]

(Goldsmith, 1976), syllable structure (Clements and Keyser, 1983), prosodic structure (Hayes, 1980), and feature geometry (Sagey, 1986). The core idea is that elements on different tiers are temporally synchronized via an association relation. Many input-output mappings are naturally characterized by rearranging those association lines across tiers (Goldsmith, 1976; Clements and Goldsmith, 1984b; Odden, 2013; Hyman, 2009).

For example, one instance of reduplication in Estako, a tonal Niger-Congo language, is shown in Figure 1. The word for 'house' in Estako is pronounced [ówà] with a high (H) tone on the first vowel and a low (L) tone on the second vowel. In Estako, reduplication conveys a repetitive meaning 'every X'. Therefore, the reduplicated form of [ówà] 'house' is [ówŏwà] 'every house', with high, rising, and low tones on the first, second, and third vowels, respectively. In ARs, segmental and tonal information are represented on separate tiers, but are coordinated in time by the association relation (shown in figures in this paper as lines). When reduplication creates a sequence of adjacent vowels V_1V_2, the first vowel V_1 delete. Its associated tone, however, does not. Instead, it reassociates to the following vowel. As shown in Figure 1, this produces a rising tone on V_2 in the output. More generally, segmental and tonal information in ARs are represented on separate tiers, but are coordinated in time by the association relation (shown in figures in this paper as lines).

1. Tone refers to the pitch level of a vowel or syllable. Languages which use pitch to mark lexical contrasts are called tonal languages. English is not a tonal language, whereas Mandarin contrasts words such as *mā* 'mother' and *mǎ* 'horse' solely by tone.

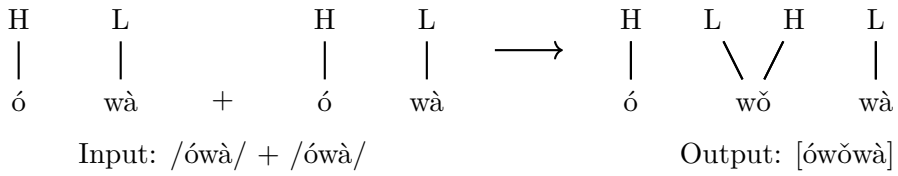


Figure 1: Association transformation in /ówà/ 'house' + /ówà/ 'house' → [ówǒwà] 'every house'. Deletion of the stem-final vowel triggers association change.

A large body of theoretical work on African tonal processes has demonstrated the advantages of autosegmental graphs (Clements and Goldsmith, 1984a; Odden, 2013; Hyman, 2009). Computationally, using multi-tiered structures has been shown to reduce the complexity of tonal patterns (Jardine, 2016; Chandlee and Jardine, 2021a). Previous work has modeled these structures using finite-state approaches (Wiebe, 1992; Bird and Ellison, 1994; Yli-Jyrä, 2013, 2015; Rawski and Dolatian, 2020). These approaches typically convert structured representations into strings or synchronized tapes to leverage finite-state machines, or rely on hardcoding different types of associations into strings.

In contrast, we propose an approach that learns how the association relation transforms directly over autosegmental graphs. As a preliminary study, we restrict our attention to tonal processes involving a single deassociation and/or reassociation operation.

We adopt the model-theoretic logical transduction framework of Yolyan (2025a). Yolyan synthesizes insights from from graph transductions (Courcelle, 1994; Engelfriet and Hoogeboom, 2001; Courcelle and Engelfriet, 2012) with learning of classes of formal languages using model-theoretic representations (Strother-Garcia et al., 2016; Chandlee et al., 2019) to learn a class of string-to-string functions. The transductions specify, via a collection of logical formulas, how to construct an output structure from a given an input structure. Yolyan searches for a minimal distinguishing structure based on the Bottom-Up Factor Inference Algorithm (BUFIA; Rawski 2021; Li 2025; Swanson et al. 2026; Li and Heinz to appear). BUFIA searches a partially ordered space of model-theoretic structures to identify a minimal hypothesis consistent with the data. For transformation learning, Yolyan (2025a,b) uses this idea to find a minimal structure which distinguishs positive examples (those that undergo the change) from negative examples (those that do not).

This work brings together these representational and learning perspectives. Using a case study of tonal contour formation (as shown in Figure 1), we show how logical transductions over autosegmental graphs can be learned using a BUFIA-style learner. In contrast to Yolyan's previous work on string functions, the target of learning here is how association relations change. While the case study is drawn from tonal phonology, the same approach can in principle be extended to other multi-linear structures. To our knowledge, the learnability of such transformations has not previously been investigated.

2. Preliminaries

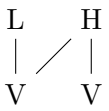
This section provides background on autosegmental graphs, matrix and logical representations thereof. We assume familiarity with first order logic.

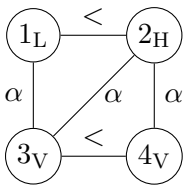
ARs represent the idea that different aspects of linguistic structure are organized on distinct levels while progressing simultaneously over time. Each tier encodes a particular type of information. In this work, we assume a two-tiered structure, though the definition remains the same in the general case.

We define autosegmental representations using model-theoretic relational structures (Jardine, 2017). A relational structure contains a domain D and instantiates a set of relations over that domain. The set of relations is called the *signature* of the model. In this paper, the signature of the model contains the unary relations {L, H, C, V} indicating which domain elements are low tones, high tones, consonant and vowels. The low and high tones make up one tier and consonant and vowels make up the other tier. The signature also contains two binary relations: $<$ is the precedence relation and α is the association relation. Well-formed autosegmental representations obey the following constraints.

- The domain is partitioned into two sets: T (tonal tier) and S (segmental tier).
- Every element in T satisfies exactly one of L or H and nothing else.
- Every element in S satisfies exactly one of C or V and nothing else.
- If $<(x, y)$ is satisfied then either $x \in T \wedge y \in T$ or $x \in S \wedge y \in S$. Also, for each tier, the precedence relation produces a total ordering over it.
- If $\alpha(x, y)$ is satisfied then either $x \in T \wedge y \in S$ or $y \in T \wedge x \in S$. The association relation is symmetric (though symmetry is not shown in Figures or logical formulas for readability).
- For all $(x, y), (x', y') \in \alpha, x < x' \Rightarrow (y < y')(y = y')$.

In short, an AR is a graph, whose vertices are partitioned into two disjoint sets, with each tier linearly-ordered, whose elements are coordinated via a temporally well-behaved association relation.[2] Figure 2 shows an AR on the left, its corresponding model-theoretic structure on the right, and the graph of the model in the center. Henceforth, we use the words 'node' and 'domain element' interchangeably.





$D = \{1, 2, 3, 4\}$
$L = \{1\} \quad H = \{2\}$
$C = \varnothing \quad V = \{3, 4\}$
$\sigma = \{3, 4\}$
$< = \{\langle 1, 2\rangle, \langle 3, 4\rangle\}$
$\alpha = \{\langle 1, 3\rangle, \langle 2, 4\rangle, \langle 2, 3\rangle\}$

Figure 2: Three representations of a two-tier autosegmental graph.

The *size* of an AR is defined as the size of the domain plus the size of the unary relations and half the size of the association relation. We leave out the precedence relation because its size is fixed given the lengths of each tier. We divide by two because *alpha* is symmetric. Formally, the size of an AR is $|D| + |H| + |L| + |C| + |V| + |\alpha|/2$. Thus, the size of the AR in Figure 2 is 7.

2. It is straightforward to generalize ARs to include n-partite graphs with n tiers.

The successor relation $\lhd$ is the transitive reduction of $<$: $x \lhd y$ if and only if $x < y \wedge \neg(\exists z)[x < z < y]$. Two nodes x, y are *neighbors* if $x \lhd y \vee y \lhd x$.

Just as it is convenient to view ARs as graphs, it is also useful to represent them with *adjacency matrices*, which use binary values to denote whether there is an edge between two nodes. For example, Table 1 shows an adjacency matrix equivalent to the AR in Figure 2.

	L	H
V_1	1	1
V_2	0	1

Table 1: Adjacency matrix representation of the ASG in Figure 2.

Formally, an AR $\mathcal{A} = \langle D, L, H, C, V, \alpha, \lhd \rangle$ can be represented by

$$M_\alpha[x, y] = \begin{cases} 1 & \text{if } (x, y) \in \alpha, \\ 0 & \text{otherwise,} \end{cases}$$

The motivation for adopting matrices is convenience. It will be important to identify structures contained within other structures. When working with strings, notions of subsequence and substring are well-defined. While Chandlee et al. (2019) and Rogers and Lambert (2019) define parallel notions for arbitrary relational structures, which can also be applied to ARs, we find it simpler to develop these notions for ARs in the context of matrices. If A is an $m \times n$ matrix then B is a *submatrix* of A if it is formed by taking $(1 \leq k \leq m)$ contiguous rows and ℓ $(1 \leq \ell \leq n)$ contiguous columns from A.[3]

An $m \times n$ matrix A is contained in a matrix B, denoted $A \sqsubseteq B$, if there exists an $m \times n$ submatrix of B, denoted B', such that for all $i \in [1, m]$ and $j \in [1, n]$, $b'_{ij} \geq a_{ij}$. **[don't the labels have to match too?]** For example, consider the structure given by the formula $\alpha(1, 2) \wedge H(1) \wedge V(2)$. This structure is contained within the Figure 2.

A matrix A is a *subfactor* of B if and only if A is contained in B. Conversely, B is a *superfactor* of A. Furthermore, if the size difference between a superfactor and its subfactor is exactly one, the superfactor is called the *next superfactor* of that subfactor.

There is a general connection between connected structures, logical formulas, and containment. For example, the structure in Figure 2 corresponds to the logical formula

$$\exists w, x, y, z\big(L(w) \wedge H(x) \wedge V(y) \wedge V(z) \wedge w < x \wedge y < z \wedge \alpha(w, y) \wedge \alpha(x, z) \wedge \alpha(x, y)\big).$$

When one considers which structures satisfy this formula, they all contain a structure isomorphic to the one in in Figure 2.

Table 2 shows matrix, graphical, and logical representations of different structures where the the variables x, y (which are in scope by the predicate $\mathsf{struc}(x, y)$. Furthermore, pairs of adjacent rows in Table 2 stand in the next superfactor relationship. The structure denoted by $\alpha(x, y) \wedge H(y)$ is a next superfactor of $\alpha(x, y)$ and so on. The 'Operation' column indicates the change that was made to obtain the next superfactor from the structure in the previous row.

3. The term "submatrix" is generally defined by deleting arbitrary rows and/or columns from the original matrix. However, in our case, we require the remaining rows and columns must be *contiguous* in the original matrix, which is related to the notion of *connectedness* in Chandlee et al. (2019).

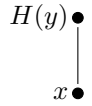
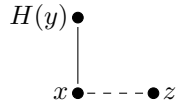
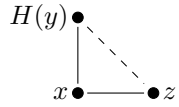
Operation	**Matrix**	**Graph**	**Logic** $\mathsf{struc}(x, y)$	**Size**
Initial	$\begin{array}{c\|c} & x \\ \hline y & 1 \end{array}$		$\alpha(x, y)$	3
Add label	$\begin{array}{c\|c} & H \\ \hline y & 1 \end{array}$		$\alpha(x, y) \wedge H(y)$	4
Add node	$\begin{array}{c\|cc} & H & z \\ \hline y & 1 & \end{array}$		$\alpha(x, y) \wedge H(x) \wedge \exists z \lhd (x, z)$	5
Add association	$\begin{array}{c\|cc} & H & z \\ \hline y & 1 & 1 \end{array}$		$\alpha(x, y) \wedge H(x) \wedge \exists z(\lhd(x, z) \wedge \alpha(y, z))$	6

Table 2: Examples of operations used in generating NextSupFact hypotheses across matrix, graph, and logical representations.

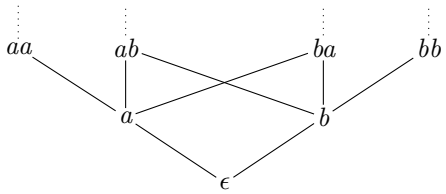


Figure 3: Strings over $\Sigma = \{a, b\}$ ordered by the containment relation (non-exhaustive)

This containment relation provides a *partial ordering* over the space of ARs. We illustrate this partial order with strings in Figure 3. A superfactor is connected to its subfactors, which are positioned below it in the diagram. The partial ordering in turn leads to entailment relations. If a string contains some factor s, then it contains all subfactors of s. Similarly, if it does not contain s, then it does not contain any superfactor of s. Readers are referred to related work (Chandlee et al., 2019; Rawski, 2021; Swanson, 2025; Swanson et al., 2026; Payne, 2024; Li, 2025) for a more detailed discussion of the partial ordering itself. Here, we emphasize that this partial order forms the foundation of bottom-up learning algorithms, since it provides a principled way to navigate the hypothesis space through entailment relations between factors.

3. Graph Transductions

We model a phonological process with graph transductions from input structures to output structures (Courcelle, 1994; Engelfriet and Hoogeboom, 2001; Courcelle and Engelfriet, 2012). Such transductions have been used to model phonological processes (Strother-Garcia, 2018; Dolatian, 2020; Chandlee and Jardine, 2021b), as well as to study different representations (Strother-Garcia, 2019; Oakden, 2020; Jardine et al., 2021; Danis, to appear).

Output		Input	Interpretation
ϕ_{Domain}	:=	TRUE	Preserve all input elements
COPY	:=	1	Define Output domain the same size as Input
$\phi_P(x)$	:=	$P(x)$	Preserve elements and their labels
$\phi_{\lhd}(x,y)$	$\approx$	$\lhd(x,y)$	Preserve linear order within tiers
$\phi_\alpha(x,y)$	:=	$\big(\alpha(x,y) \wedge \neg\mathsf{deassoc}(x,y)\big) \vee \big(\neg\alpha(x,y) \wedge \mathsf{reassoc}(x,y)\big)$	Update association relations
$\phi_{\mathsf{license}}(x)$	:=	$\exists y\, \phi_\alpha(x,y)$	Preserve only associated elements

Table 3

In this framework, a set of logic formulas define a function from ARs to ARs. The formulas are always evaluated with respect to the input structure. Whether they evaluate to true or false has consequences for the structure of the output.

- The domain formula ϕ_{Domain} determines which ARs are in the domain of the function. Here, we set this value to True, indicating this function is total.
- The copy set helps determine the size of the output structures. Here, we set it to 1, which means every node in the output corresponds to one element in the input.
- For each unary relation $P \in \{L, H, C, V\}$, the formula $\phi_P(x)$ determines whether node x in the output has property P. Here, nodes in the output have the same properties as their corresponding nodes in the input.
- The formula $\phi_<(x,y)$ determines which pairs of nodes in the output structure stand in the precedence relation. Here, pairs of nodes in the output stand in the precedence relation only if their corresponding nodes in the input are in the precedence relation.
- The formula $\phi_\alpha(x,y)$ determines which pairs of nodes in the output structure stand in the association relation. Here pairs of nodes stand in the association relation provided (i) they were associated in the input and they do not satisfy deassoc (deassociation), or (ii) they were not associated in the input and they do satisfy reassoc (reassociation). The predicates deassoc and reassoc are the logical predicates that vary on a language specific basis. These are the parameters that have to be learned.
- The licensing formula $\phi_{\mathsf{license}}(x)$ determines which nodes in the output are liscensed. According to the semantics of graph transductions, unlicensed nodes disappear, along with any relations associated with them. Here, only output nodes that stand in an association relation in the output are liscensed.

Table 3 summarizes these formulas.

As shown in Table 3, the transduction defines an output model with the same domain as the input structure, while filtering out unassociated elements by $\phi_{\mathsf{license}}(x)$. All unary relations $P(x)$ in the input signature are preserved in the corresponding output. Similarly, precedence relations are preserved. The only change is the association relation α. In this way, the problem of learning association-line transformations can be reduced to learning the logical formulas for deassoc and/or reassoc. When no transformation occurs between the input and output representations, neither deassoc nor reassoc takes place (i.e., both have

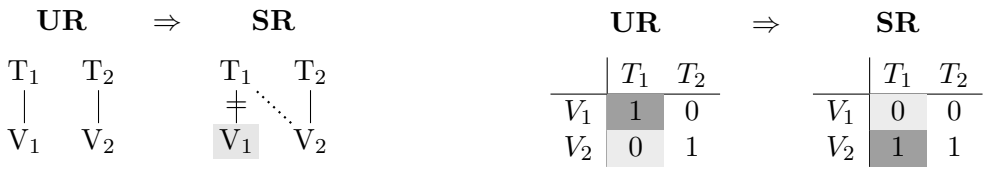


Figure 4: Contour formation in both graph-theoretic and matrix representations.

the value of false). The association relation therefore remains unchanged, and all input associations are faithfully preserved in the output.

Since these two predicates are combined through disjunction, two parallel learning processes apply simultaneously, and their outputs are combined to define the final association relation α'.

To illustrate, consider contour formation as shown in Figure 1. Figure 4 visualizes this process in both graph-theoretic and matrix representations. From a logical perspective, this transformation is captured by three predicates: $\mathsf{deassoc}$, $\mathsf{reassoc}$, and ϕ_{license}. The $\mathsf{deassoc}$ function identifies the association to be broken between T_1 and the deleted vowel V_1 (the crossed-out line). The conjunction $\alpha(x, y) \wedge \neg\mathsf{deassoc}(x, y)$ removes this association (T_1, V_1) while preserving all unaffected associations, that is, (T_2, V_2). The $\mathsf{reassoc}$ function then links T_1 to the surviving vowel V_2 (the dotted line). As a result, V_1 is no longer associated with any element on the opposite tier and therefore fails to satisfy the licensing condition ϕ_{license} in the output. Consequently, V_1 is not licensed and is omitted from the surface form (shown in gray).

The $\mathsf{deassoc}$ and $\mathsf{reassoc}$ operations are equivalently reflected in the adjacency matrices in Figure 4. The value for (T_1, V_1) changes from 1 (in the input) to 0 (in the output), which indicates delinking; the value corresponding to (T_1, V_2) changes from 0 to 1, which indicates reassociation. Formally, the set of deassociated pairs and the set of reassociated pairs can be identified by $\{(i, j) \mid \mathrm{SR}_{ij} < \mathrm{UR}_{ij}\}$ and $\{(i, j) \mid \mathrm{UR}_{ij} < \mathrm{SR}_{ij}\}$, respectively.

4. Association Learning

This section follows the ideas of using logical transductions to learn phonological mappings in Yolyan (2025a), who in turn leveraged the entailment relation in the partially ordered space crucial to Chandlee et al. (2019) to find a smallest environment that triggers a change.

Importantly, if an environment (which itself is a structure, henceforth we call it 'environment') is sufficient to trigger a transformation, then any superfactor containing that environment will likewise exhibit the same transformation. On the other hand, if an environment fails to trigger the change, then all of its subfactors are likewise insufficient to trigger the change. In other words, given a hypothesized environment, all data satisfying this condition are expected to undergo the transformation (positive evidence), while all data not satisfying this condition are expected not to undergo the transformation (negative evidence). If some positive data fail to undergo the transformation, then the hypothesis is too restrictive. Conversely, if some negative data nevertheless undergo the transformation, then the hypothesis is too general. The goal of learning is therefore to identify the smallest

Data: $D = \{(I_1, O_1), \ldots, (I_n, O_n)\}, D^+ = \{\, I_n \mid I_n \neq O_n \,\}, D^- = \{\, I_n \mid I_n = O_n \,\}, k$

$P \leftarrow \emptyset$
$Q \leftarrow c_0$
while $Q \neq \emptyset$ **do**
  $c \leftarrow Q$.Dequeue()
  if $\forall g \in V^+,\ c \sqsubseteq g$ **then**
    if $\forall g \in V^-,\ c \not\sqsubseteq g$ **then**
      return c
    else
      foreach $s \in \textsc{NextSupFact}(c)$ *such that* $|s| < k \wedge (\forall p \in P)[s \not\sqsubseteq p]$ **do**
        Q.Enqueue(s)
  else
    $P \leftarrow P \cup \{c\}$
return $\perp$

Algorithm 1: Learning Association Changes

common environment that is contained in all positive evidence, but absent in all negative evidence.

Yolyan (2025a) worked with string-to-string functions. However, in autosegmental phonology, environments are expressed as structural relations over graphs. Because model-theoretic representations generally form a partial order with the containment relation (Chandlee et al., 2019), the same ideas apply to ARs. Equivalently, transformations can be viewed as changes to the graph's adjacency matrix, where an entry changes from $1 \rightarrow 0$ or $1 \rightarrow 0$.

The learning algorithm is shown in Algorithm 1. Initially, the learner identifies changing association sites by comparing the input and output association matrices. If a deassociation is found, the learner then initializes the search with the corresponding most general logical hypothesis: $\alpha(x, y)$. If instead a reassociation is found, then the learner starts with $neg\alpha(x, y)$. We treat deassociation and reassociation as two independent processes and combine the resulting hypotheses through disjunction.

The input data consists of a finite set of input-output ASG pairs, which can be partitioned into positive and negative evidence, denoted D^+ and D^- respectively. It also requires a bound k that limits the graph size.

The algorithm conducts a breadth-first search over a partially ordered hypothesis space. At each step, a candidate hypothesis c is first tested against the positive evidence: if c is not contained in all positive examples, then the current hypothesis is incompatible with the observed transformation and thus is added to the pruning set P. Otherwise, the learner checks c against all negative examples: if c is NOT contained in any datum in V^-, then the current environment is output as it is consistent with all data. Otherwise, the hypothesis is considered too general, and will be updated minimally using NextSupFact. Meanwhile, the learner filters out candidate environments that contain pruned structures or exceed the predetermined complexity bound k, so as to keep the search space tractable. Since hypotheses are explored breadth-first, the first valid hypothesis returned by the learner is guaranteed to be the minimal structural environment consistent with all positive and negative evidence.

In terms of NextSupFact, there are many possible ways to generate superfactors while still covering the same hypothesis space. Here, we propose one method that explicitly connects graph-theoretic representations with their logical counterparts.

Recall the size of an ASG as the total number of nodes and association lines. In addition, the next superfactors of a structure are those whose size increases minimally by 1. Consequently, increases in size can only arise through the introduction of new elements or new association relations.

Therefore, when generating next superfactors, the learner preferrs the smallest possible structure and expands the domain only when required by the data. Specifically, see the following order:

(i) labeling unspecified nodes; if labeling does not contribute to the transformation, then backtracks to the under-specified variable.[see note below]
(ii) introducing a neighboring node, which may either follow or precede an existing node on either tier, at both boundaries;
(iii) adding association lines, provided that both endpoints are already present in the structure.

Table 2 presents a non-exhaustive set of examples illustrating the generation of NextSupFact from an initial deassoc hypothesis.

As can be seen, each operation provides an equivalent interpretation from matrix, graph-theoretic, and logical perspectives. New quantifiers are only introduced through successor relations, corresponding to the addition of neighboring nodes in the graph structure. Within the same quantifier scope, the learner may further specialize the hypothesis through conjunction, including adding association relations (corresponding to changing matrix cells) or specifying node labels and feature values.

5. Case Study: Contour Formation

We use the contour formation to demonstrate both the logical transduction framework and BUFIA-style learning over ASGs. We assume a model $\mathcal{M} = \{H, L, C, V, \alpha, \lhd\}$. where H and L are tonal elements on the tone tier T, while C and V belong to the segmental tier S. Table 4 gives five input-output pairs, and cells marked with a dash indicate the same output as the input. The headers provide shorthand notations for each pair. The contour formation process can be observed in the CVV-LH and CVV-HL examples, where both tones become associated with the same vowel. This transformation can equivalently be represented as a change in the association matrix. Notably, tonal identities themselves are irrelevant to the deassociation and reassociation process: both CVV-LH and CVV-HL undergo contour formation once the two tones are linked to a single vowel. By contrast, an LH sequence does not necessarily surface as a contour; whether it does depends on the structural configuration of the vowels. For example, the LH sequence surfaces as a contour in CVV-LH, where the vowels are adjacent, but not in VCV-LH, where they are separated.

The learner first partitions the data into positive evidence V^+ and negative evidence V^-. In this toy example, V^+ consists of CVV-LH and CVV-HL, and V^- consists of the rest pairs marked with dashes in Table 4. By comparing V^+, the learner can thus observe both

	CV-H	CV-L	VCV-LH	CVV-LH	CVV-HL
UR	H | C V	L | C V	L H | | V C V	L H | | C V_1 V_2	H L | | V_1 V_2
SR	–	–	–	L H \ | C V_2	H L \ | V_2

Table 4: Toy ASG

$$A_i = \begin{array}{c|cc} & T & T \\ C & 0 & 0 \\ V & 1 & 0 \\ V & 0 & 1 \end{array} \quad \Rightarrow \quad A_o = \begin{array}{c|cc} & T & T \\ C & 0 & 0 \\ V & 0 & 0 \\ V & 1 & 1 \end{array}$$

Table 5: Input-output matrices (A_i) and (A_o) contour formation. The highlighted entries indicate changes in association relations.

deassociation and reassociation occur. Here, we will only demonstrate the deassociation learning since the reassociation proceeds analogously.

For deassociation, the learner then begins with the following initial hypothesis, which states that any association line present in the input is deleted in the output.

$$H_0 : \mathsf{deassoc}(x, y) \quad = \alpha(x, y)$$

The following step is to test H_0 against both positive and negatice evidence. Trivailly, it is accepted by all positive examples, but, when evaluated against the negative evidence, the hypothesis is rejected: in CV-H, CV-L, and VCV-LH, the association lines in the input remain associated in the output, contrary to the prediction of H_0. The learner therefore concludes that the hypothesis is too general and must be made more specific.

To refine the hypothesis, the learner generates increasingly restricted structures by introducing additional structural conditions over the ASG. Following a label-node-association order, the learner first constructs next superfactors by specializing the hypothesis through node labeling. Naively, the learner iterates over all symbols available on two corresponding tiers and substitutes them into the existing unlabeled nodes. The resulting factors are shown in Table 6: (6a) and (6b) label y as H or L on the tone tier, while (6c) and (6d) label x as V or C on the segment tier.

When evaluating the candidate hypotheses in Table 6, we find that neither (6a) nor (6b) covers all positive examples. This indicates that tonal identity does not contribute to the transformation, and the learner therefore backtracks to the underspecified node. **[It seems like you are doing more here, and throughout, than algo 1. In particular, even though none of th 6a-d are consistent with the data, you differentiate between 6ab and 6cd. Algo 1 doesn't. Let's discuss.]** Likewise, on the segmental tier, (6d)

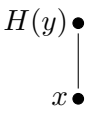
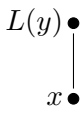
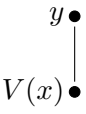
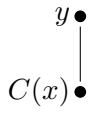
	(6a)	(6b)	(6c)	(6d)
Label Nodes				
Contained by all V^+	×	×	✓	×
Absent in V^-	×	×	×	×

Table 6: Next Superfactors of H_0

is inconsistent with the positive evidence and is rejected, whereas (6c) remains consistent with all positive examples. The learner therefore refines the hypothesis as follows:

$$H_1: \quad \mathsf{deassoc}(x, y) = \alpha(x, y) \wedge V(x)$$

However, since (6c) is also attested in negative examples, it fails to distinguish contexts in which the transformation applies from those in which it does not. The learner therefore generates the next superfactors of (6c) by introducing adjacent nodes on either tier. As the previous step established that tonal identity is not relevant to the transformation, node y remains underspecified rather than being assigned a tonal label. The resulting factors are shown in Table 7, where (7a)–(7b) introduce a new node that immediately precedes or follows the existing node on the segment tier, while (7c)–(7d) do so on the tone tier.

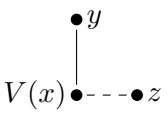
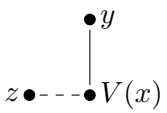
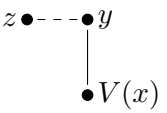
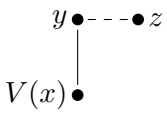
	(7a)	(7b)	(7c)	(7d)
				
Contained by all V^+	✓	×	×	✓
Absent in all V^-	×	×	×	×

Table 7: Next Superfactors of (6c) in H_1

Again, the following step is to test against V^+ and V^-. In this round, two candidates, namely (7a) and (7d), are contained by all positive evidence. At the same time, the learner rejects (7b) and (7c) and prunes them from hypothesis space. This suggests that the current deassociation process is insensitive to elements on the left side of the deletion site, whether on the tonal tier or the segmental tier, but depends on rightward context. Pruning off (7b) and (7c) eliminates all subsequent hypotheses that introduce leftward contextual conditions but only restricts the search space to rightward environments only.

Based on candidates (7a) and (7d), the learner has the following updated hypotheses:

$$H_{2a}: \quad \mathsf{deassoc}(x, y) = \alpha(x, y) \wedge V(x) \wedge \exists z(x \lhd z)$$

$$H_{2d}: \quad \mathsf{deassoc}(x, y) = \alpha(x, y) \wedge V(x) \wedge \exists z(y \lhd z)$$

However, both remaining candidates are still present in some negative examples and therefore remain too general and need further refinement. At this stage, the learner could introduce additional nodes or add associations between z and a node on the opposite tier, but either operation would increase the size of the hypothesis. Labeling, by contrast, does not increase complexity. Consequently, the learner specializes the newly introduced node z. The resulting factors are shown in Table 8. Factors (8a)–(8b) refine H_{2a} by labeling z as V or C on the segmental tier, whereas (8c)–(8d) label z as H or L on the tonal tier.

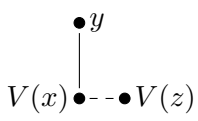
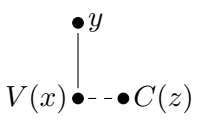
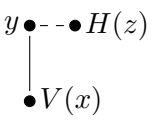
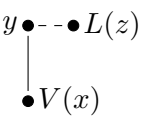
	(8a)	(8b)	(8c)	(8d)
				
Contained by all V^+	✓	×	×	×
Absent in V^-	✓	×	×	×

Table 8: Next Superfactor of H_{2a} and H_{2d}

Only candidate (8a) is both contained in all positive evidence and absent from all negative evidence. The learner therefore converges on this hypothesis as a minimal structure that triggers the transformation and is consistent with both positive and negative evidence. This formula encodes the same structural condition illustrated in Figure 4.

$$\mathsf{deassoc}(x, y) = \alpha(x, y) \wedge V(x) \wedge \exists z\big(x \lhd z \wedge V(z)\big)$$

Importantly, one may observe that triggering deassociation does not require T_2 to be associated with V_2. The learned hypothesis therefore predicts that deassociation applies whenever two vowels are adjacent, regardless of whether the following vowel is associated with a tone.

The $\mathsf{reassoc}$ inference follows the same process but with a different starting hypothesis

$$\mathsf{reassoc}(x, y) = \neg\alpha(x, y)$$

In this process, the triggering context lies to the left of the reassociation site, in contrast to the deassociation case. The learner eventually converges on the following hypothesis:

$$\mathsf{reassoc}(x, y) = \neg\alpha(x, y) \wedge V(x) \wedge \exists z\big((z \lhd x) \wedge V(z) \wedge \alpha(z, y)\big)$$

We combine the two operations into the final transformation.

$$\begin{aligned}\phi_{\alpha(x,y)} := {} & \alpha(x, y) \wedge V(x) \wedge \exists z\big((x \lhd z) \wedge V(z)\big) \\ & \vee \neg\alpha(x, y) \wedge V(x) \wedge \exists z\big((z \lhd x) \wedge V(z) \wedge \alpha(z, y)\big)\end{aligned}$$

Lastly, the licensing function ϕ_{license} filters out elements that are not associated in the surface representation. Such unlicensed elements are not realized phonetically and therefore do not appear in the output.

6. Conclusion

This work has argued that phonological processes that involve multi-tiered structures such as autosegmental representations can be uniformly captured using model theory and logical transductions. The input-output mapping can be modeled by modifying a single relation, α, through two operations of deassoc and reassoc. We also show that BUFIA-style learning can be naturally extended to such enriched representations by traversing a partially ordered hypothesis space. A crucial component is NextSupFactor, which generates minimally more complex structures through node labeling, the introduction of neighboring nodes, and the addition of association relations, to keep the search space tractable. The resulting hypothesis is a structural environment, or equivalently a logical formula, that is consistent with both the positive and negative evidence.

An important direction for future work is to relax the current restriction to single deassociation and reassociation processes. Doing so will require learning more complex hypotheses involving conjunctions of structural conditions. Another limitation of the current analysis is that the transduction relies on quantification to capture across-tier dependencies. Future work can determine whether these processes can be simplified in a quantifier-free (QF) process. This opens up more discussions about how linguistically meaningful representations can facilitate grammar inference.

References

Steven Bird and T Mark Ellison. One-level phonology: Autosegmental representations and rules as finite automata. *Computational Linguistics*, 20(1):55–90, 1994.

Jane Chandlee and Adam Jardine. Input and output locality and representation. *Glossa: a journal of general linguistics*, 6(1), 2021a.

Jane Chandlee and Adam Jardine. Computational universals in linguistic theory: Using recursive programs for phonological analysis. *Language*, 93:485–519, 2021b.

Jane Chandlee, Rémi Eyraud, Jeffrey Heinz, Adam Jardine, and Jonathan Rawski. Learning with partially ordered representations. In *Proceedings of MOL16*, pages 91–101, 2019.

George Clements and Jay Keyser. *CV phonology: a generative theory of the syllable.* Cambridge, MA: MIT Press, 1983.

George N Clements and J Goldsmith. Autosegmental studies in bantu tone, 1984a.

George N Clements and John Goldsmith. Autosegmental studies in bantu tone: Introduction. *Clements & Goldsmith (1984b)*, 117, 1984b.

Bruno Courcelle. Monadic second-order definable graph transductions: a survey. *Theoretical Computer Science*, 126:53–75, 04 1994.

Bruno Courcelle and Joost Engelfriet. *Graph Structure and Monadic Second-Order Logic, a Language Theoretic Approach.* Cambridge University Press, 2012.

Nick Danis. Logical transductions are not sufficient for notational equivalence. In *Proceedings of the 2024 Annual Meeting of Phonology*, to appear.

Hossep Dolatian. *Computational locality of cyclic phonology in Armenian*. PhD thesis, Stony Brook University, 2020.

Joost Engelfriet and Hendrik Jan Hoogeboom. MSO definable string transductions and two-way finite-state transducers. *ACM Transactions on Computational Logic*, 2(2):216–254, April 2001. ISSN 1529-3785.

John Anton Goldsmith. *Autosegmental phonology*. PhD thesis, Massachusetts Institute of Technology, 1976.

Bruce Philip Hayes. *A metrical theory of stress rules*. PhD thesis, Massachusetts Institute of Technology, 1980.

Larry M Hyman. The representation of tone. *UC Berkeley PhonLab Annual Report*, 5(5), 2009.

Adam Jardine. Computationally, tone is different. *Phonology*, 33(2):247–283, 2016.

Adam Jardine. The local nature of tone-association patterns. *Phonology*, 34(2):363–384, 2017.

Adam Jardine and Jeffrey Heinz. A concatenation operation to derive autosegmental graphs. In *Proceedings of the 14th Meeting on the Mathematics of Language (MoL 2015)*, pages 139–151, 2015.

Adam Jardine, Nick Danis, and Luca Iacoponi. A formal investigation of Q-theory in comparison to autosegmental representations. *Linguistic Inquiry*, 52(2):333–358, 2021. doi: 10.1162/ling_a_00376.

Han Li. Learning tonotactic patterns over autosegmental representations. In *Proceedings of the Annual Meetings on Phonology*, volume 1. University of Massachusetts Amherst Libraries, 2025.

Han Li and Jeffrey Heinz. Learning tonotactics with autosegmental representations. *Phonology*, to appear.

Chris Oakden. Notational equivalence in tonal geometry. *Phonology*, 37(2):257–296, 2020. doi: 10.1017/S0952675720000123.

David Odden. *Introducing Phonology*. Cambridge University Press, Cambridge, 2 edition, 2013.

Sarah Payne. A generalized algorithm for learning positive and negative grammars with unconventional string models. In *Proceedings of the Society for Computation in Linguistics 2024*, pages 75–85, 2024.

Jonathan Rawski. *Structure and learning in natural language*. PhD thesis, State University of New York at Stony Brook, 2021.

Jonathan Rawski and Hossep Dolatian. Multi-input strictly local functions for tonal phonology. In *Proceedings of the Society for Computation in Linguistics 2020*, pages 208–223, 2020.

James Rogers and Dakotah Lambert. Some classes of sets of structures definable without quantifiers. In *Proceedings of the 16th Meeting on the Mathematics of Language*, pages 63–77, Toronto, Canada, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/W19-5706. URL https://www.aclweb.org/anthology/W19-5706.

Elizabeth Caroline Sagey. *The representation of features and relations in non-linear phonology*. PhD thesis, Massachusetts Institute of Technology, 1986.

Kristina Strother-Garcia. Imdlawn Tashlhiyt Berber syllabification is quantifier-free. In *Proceedings of the Society for Computation in Linguistics*, volume 1, 2018. Article 16.

Kristina Strother-Garcia. *Using model theory in phonology: a novel characterization of syllable structure and syllabification*. University of Delaware, 2019.

Kristina Strother-Garcia, Jeffrey Heinz, and Hyun Jin Hwangbo. Using model theory for grammatical inference: a case study from phonology. In Sicco Verwer, Menno van Zaanen, and Rick Smetsers, editors, *Proceedings of The 13th International Conference on Grammatical Inference*, volume 57 of *JMLR: Workshop and Conference Proceedings*, pages 66–78, October 2016.

Logan Swanson. Learning multi tier-based strictly 2-local languages. In *Proceedings of the 18th Meeting on the Mathematics of Language*, pages 33–43, 2025.

Logan Swanson, Jeffrey Heinz, and Jonathan Rawski. Phonotactic learning with structure, not statistics. *Linguistic Inquiry*, pages 1–24, 05 2026.

Bruce Wiebe. Modelling autosegmental phonology with multi-tape finite state transducers. 1992.

Anssi Yli-Jyrä. On finite-state tonology with autosegmental representations. In *Proceedings of the 11th international conference on finite state methods and natural language processing*, pages 90–98, 2013.

Anssi Yli-Jyrä. Three equivalent codes for autosegmental representations. In *Proceedings of the 12th International Conference on Finite-State Methods and Natural Language Processing 2015 (FSMNLP 2015 Düsseldorf)*, 2015.

Tatevik Yolyan. A framework for learning phonological maps as logical transductions. In *Proceedings of MOL18*, pages 44–58, 2025a.

Tatevik Arayevna Yolyan. *Phonological Expressivity and Learning via Boolean Monadic Recursive Schemes*. PhD thesis, Rutgers The State University of New Jersey, School of Graduate Studies, 2025b.